

Compiling Spreadsheet Formulas to Vector Code

Ada Lindqvist

Tomas Okafor

Abstract

Large spreadsheets recalculate slowly, because every formula is interpreted cell by cell. We compile each column of formulas into a single vectorized kernel instead, and check the compiled sheet against the interpreter on every recalculation of a test corpus.

1 Introduction

Recalculating a large workbook takes most of the time its users spend waiting on it. The interpreter evaluates one cell at a time, and a column of ten thousand copies of one formula is evaluated ten thousand times. We compile such a column once, into code that evaluates it whole, and keep the interpreter as the reference it is checked against. This paragraph runs on for a few lines so that the page has a measure of its body text, as a real paper’s would.

The compiled sheets recalculate between ten and forty times faster, and agree with the interpreter on every workbook in the corpus.

A Supplementary Details

A.1 Workbook Corpus

We collected workbooks from two public archives and kept those with at least a thousand formulas. Both archives give similar speedups (Table ??), so the method does not depend on where a workbook came from.

Table 1: **Workbook corpus.** Formulas per sheet; recalculation time.

Archive	Kind	Sheets	Formulas	Time
Finance	Ledger	13	6.1k	0.05 s
	Forecast	3	5.2k [§]	0.08 s
Science	Lab log	20	69k	3.26 s
	Survey	6	69k	— [†]

[§]Volatile functions recomputed each time. [†]Not timed.

A.2 Interpreter Comparison Details

Lab log (deep dependencies). The run checked only at the end of each recalculation took forty-two attempts to find a single wrong reference. Comparing every column as it was compiled found the same reference at once, and named the formula it was in.

A.3 Hardware Comparison

Table 2: **Throughput elsewhere.** Cells per second.

Machine	Cells/s	vs. Interpreter
Laptop	13.1M	29×
Server	1.4B	23×

A.4 Compilation Metrics

Compilation time was logged for every sheet. Most columns compiled in under a millisecond, and the slowest were those that mixed text and numbers in one formula.

B Optimization Guide

The following patterns, distilled from the corpus, make a compiled sheet faster. They are ordered by how much they usually help.

1. Fixed-width columns. The compiler needs every column’s type known in advance. Replace mixed columns (numbers, text, and empty cells side by side) with typed `float64` arrays padded to the sheet’s height, and mark the empty cells with a sentinel value.

2. Branchless conditionals with `np.where`. Replace each `IF` with `np.where(condition, if_true, if_false)`, which computes both branches and selects the result by mask; this is faster on wide columns because the loop never stops to branch:

```
total = np.where(paid, amount, 0.0)
column = compile_column(
    formula,
    dtype=np.float64
)
```

3. Cache the compiled kernels. Key each kernel by its formula’s shape, so that a column copied elsewhere in the workbook reuses the code compiled for the first, and compile them once when the workbook is opened.

B.1 Recalculation Prompt

The following note is shown to whoever maintains the compiler. It says what a change may and may not do.

```
The compiled sheet agrees with the interpreter. Now make it
faster without changing any result.
Target: Fewer seconds per recalculation, every check passing.
Constraints:
- Every comparison must still pass afterwards
- Do not change the public API (open, recalc, cell
  values)
```

Algorithm 1 Column-by-column compilation.

```
1: Phase 1: Column compilation
2: for  $k = 1$  to  $K$  do
3:    $c'_k \leftarrow \text{COMPILE}(c_k)$ 
4:   if  $\text{COMPARE}(c'_k) = \text{SAME}$  then
5:     break
6:   end if
7: end for
8: return  $S_{\text{compiled}}$ 
```

Algorithm 1 sets out the loop the compiler runs, column by column, until every comparison passes.